

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет Информационных технологий
Кафедра Информатики и информационных технологий

направление подготовки
09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 16

Дисциплина: «BackEnd-разработка»

Тема: *Добавление аутентификации и авторизации в веб-приложение на основе
ASP.NET Core*

Выполнил(а): студент(ка) группы 241-333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Москва
2026

Цель работы:

Освоить процесс добавления механизмов аутентификации и авторизации в веб-приложение в ASP.NET Core.

Задание:

1. Настроить в проекте механизм JWT-аутентификации с использованием встроенных средств ASP.NET Core.
2. Реализовать выдачу JWT-токена после успешной аутентификации пользователя в приложении.
3. Добавить в JWT-токен основные данные пользователя, например идентификатор, логин и роль пользователя.
4. Реализовать проверку JWT-токена при обращении клиента к защищённым ресурсам backend-приложения.
5. Использовать атрибуты `[Authorize]` и `[Authorize(Roles = "...")]` для ограничения доступа к отдельным контроллерам или методам приложения.
6. Протестировать доступ к защищённым ресурсам без токена, с корректным токеном и с токеном пользователя, не имеющего нужной роли.
7. Проверить корректность работы механизмов аутентификации и авторизации при различных уровнях доступа пользователей.

1. Написание приложения

1. Для создания проекта использовался шаблон **ASP.NET Core Web App**.
2. Конфигурация JWT в `appsettings.json`. Для хранения настроек JWT-аутентификации была создана отдельная секция конфигурации.
3. Создание модели данных и DTO. В качестве модели пользователя была создана сущность `User` с полями: `Id`, `Username`, `PasswordHash`, `Role`. Пароли хранятся в хешированном виде с использованием алгоритма **BCrypt**. Для разделения контракта API и внутренней модели созданы две DTO-модели:
 - `LoginModel` — для приёма данных при входе
 - `RegisterModel` — для приёма данных при регистрации

4. Реализация сервиса генерации токенов. Создан отдельный сервис `TokenService`, отвечающий за формирование JWT-токенов. Сервис внедряется через DI и использует конфигурацию из `IConfiguration`. Для подписи токена используется симметричный ключ (время жизни 60 минут). Утверждения, добавляемые в токен:

- `sub` (Subject) — идентификатор пользователя;
- `jti` (JWT ID) — уникальный идентификатор токена;
- `name` — имя пользователя;
- `nameid` — идентификатор пользователя (для совместимости);
- `role` — роль пользователя (для авторизации).

5. Настройка аутентификации и авторизации в `Program.cs`. JWT-аутентификация настраивается через `AddAuthentication().AddJwtBearer()`. Ключевой объект — `TokenValidationParameters`, определяющий правила валидации токена:

Таблица 1 Параметры валидации JWT-токена

Параметр	Значение	Назначение
<code>ValidateIssuer</code>	<code>True</code>	Проверка издателя токена
<code>ValidateAudience</code>	<code>True</code>	Проверка издателя токена
<code>ValidateLifetime</code>	<code>True</code>	Проверка срока действия
<code>ValidateIssuerSigningKey</code>	<code>True</code>	Проверка подписи токена
<code>ValidIssuer</code>	<code>JwtSettings.Issuer</code>	Ожидаемый издатель
<code>ValidAudience</code>	<code>JwtSettings.Audience</code>	Ожидаемый получатель
<code>IssuerSigningKey</code>	<code>SymmetricSecurityKey</code>	Ключ для проверки подписи
<code>ClockSkew</code>	<code>TimeSpan.Zero</code>	Запрет отклонения по времени

Дополнительно настроены политики авторизации через `AddAuthorization()`:

- `AdminOnly` — только для администраторов;
- `AdminOrModerator` — для администраторов и модераторов

6. Реализация контроллеров. **`AuthController`** — публичный контроллер для аутентификации:

- POST /api/auth/login — проверка логина/пароля и выдача JWT;
- POST /api/auth/register — регистрация нового пользователя.

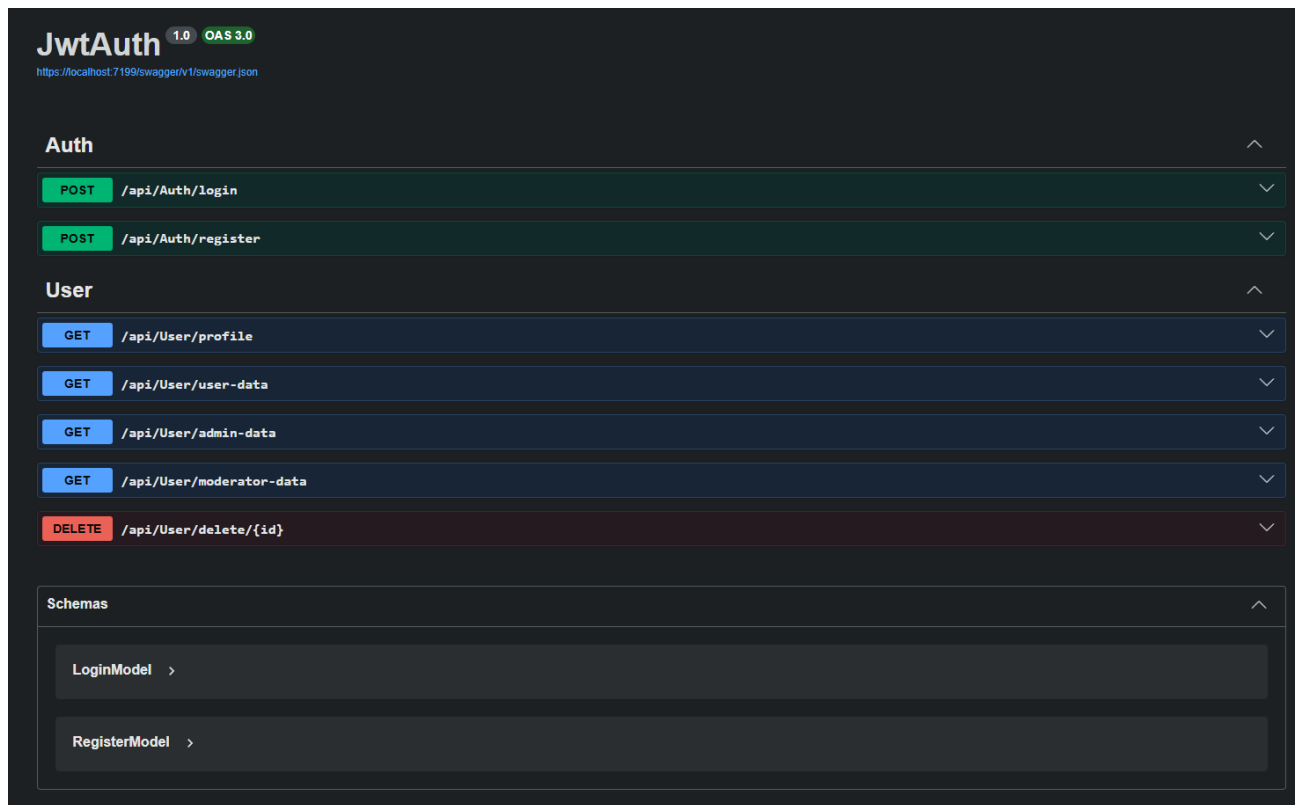
UserController — защищённый контроллер с разными уровнями доступа:

- [Authorize] — любой аутентифицированный пользователь;
- [Authorize(Roles = "User")] — только пользователи с ролью User;
- [Authorize(Roles = "Admin")] — только администраторы;
- [Authorize(Policy = "AdminOrModerator")] — по кастомной политике.

Таблица 2 Реализованные API-эндпоинты

Метод	URL	Доступ	Коды ответа
POST	/api/auth/login	Публичный	200 OK, 401 Unauthorized
POST	/api/auth/register	Публичный	200 OK, 409 Conflict
GET	/api/user/profile	[Authorize]	200 OK, 401 Unauthorized
GET	/api/user/user-data	Roles="User"	200 OK, 403 Forbidden
GET	/api/user/admin-data	Roles="Admin"	200 OK, 403 Forbidden
GET	/api/user/moderator-data	Policy="AdminOrModerator"	200 OK, 403 Forbidden
DELETE	/api/user/delete/{id}	Roles="Admin"	200 OK, 403 Forbidden

7. Для удобства тестирования была создана HTML-страница (wwwroot/index.html) с формами входа/регистрации, отображением JWT-токена и кнопками для вызова защищённых эндпоинтов. Страница раздаётся через UseStaticFiles(), а маршрут по умолчанию настроен через MapFallbackToFile("index.html").



Изображение 1 Swagger UI со списком всех эндпоинтов API

JWT Authentication

Вход

Логин:

Пароль:

Войти

Регистрация

Логин:

Пароль:

Роль:

User

Зарегистрироваться

JWT Token

Токен не получен

Тестирование доступа

Профиль

User Data

Admin Data

Moderator Data

Результат запроса

Нажмите кнопку для тестирования...

Быстрые тесты

Войти как Admin

Войти как User

Войти как Moderator

Очистить токен

Изображение 2 Минимальный UI для тестирования JWT-аутентификации

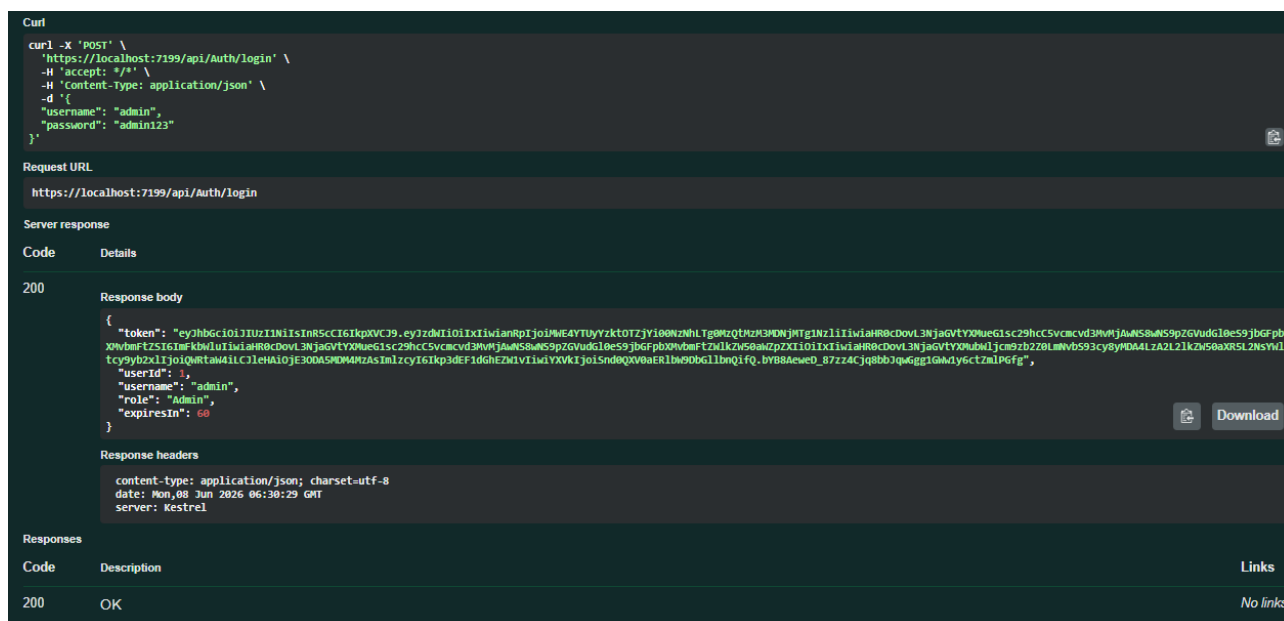
2. Запуск приложения и тестирование

Для тестирования API использовались **Swagger UI** и **минимальный HTML/JS UI**. Тестирование проводилось в три этапа:

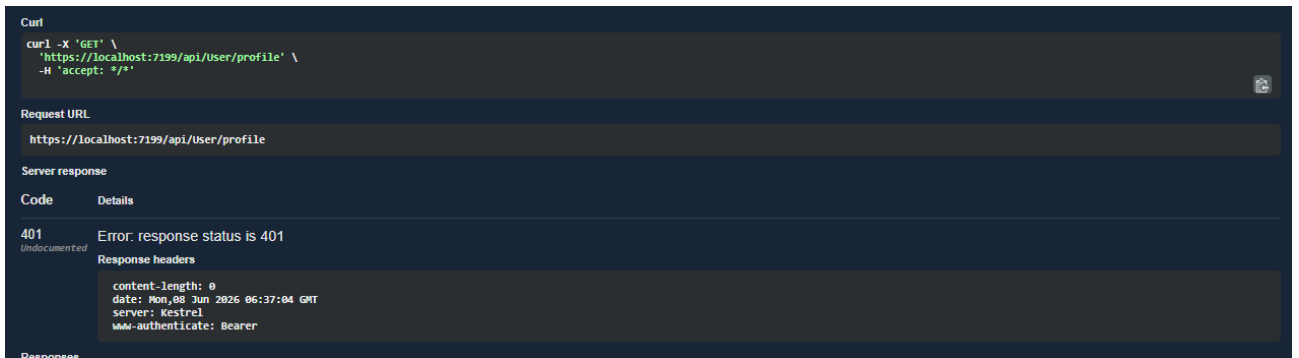
1. Получение JWT-токена через endpoint `/api/auth/login`;
2. Выполнение запросов к защищённым ресурсам с токеном в заголовке `Authorization: Bearer <token>`;
3. Проверка различных сценариев доступа (без токена, с корректным токеном, с недостаточной ролью).

Таблица 3 Результаты тестирования

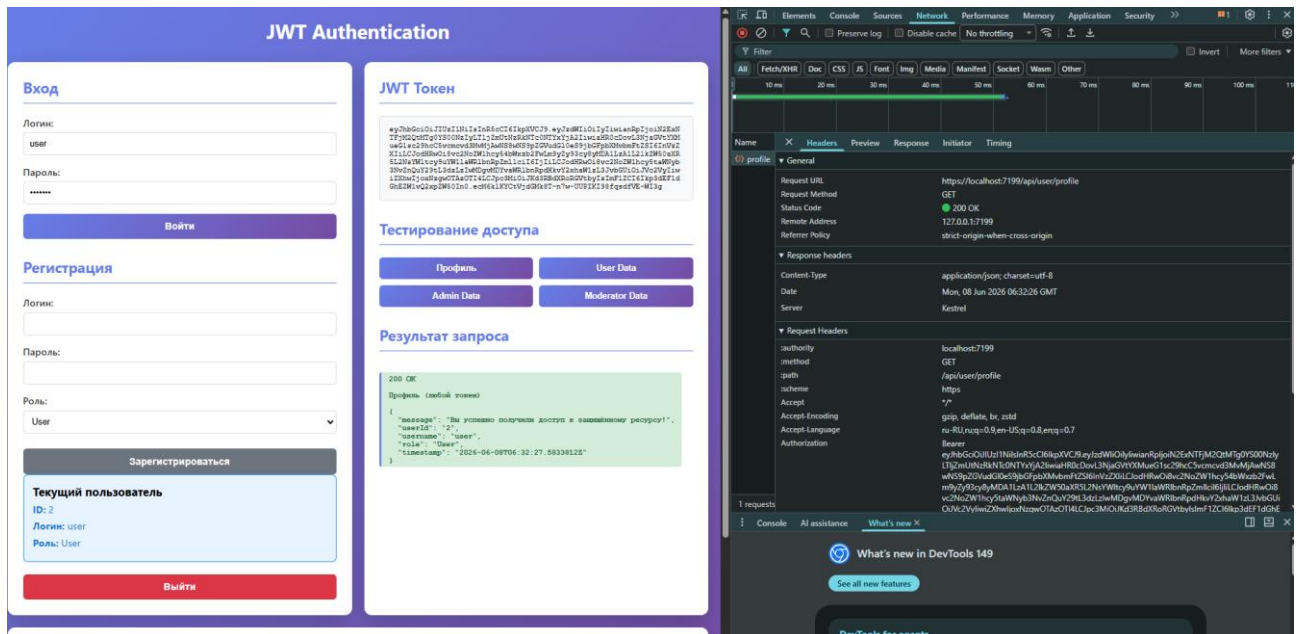
№	URL	HTTP	Параметры	Результат
1	/api/auth/login	POST	{"username":"admin","password":"admin123"}	200 OK
2	/api/user/profile	GET	Без токена	401 Unauthorized
3	/api/user/profile	GET	С токеном User	200 OK
4	/api/user/admin-data	GET	С токеном User	403 Forbidden
5	/api/user/admin-data	GET	С токеном Admin	200 OK
6	/api/user/user-data	GET	С токеном Admin	403 Forbidden
7	/api/user/moderator-data	GET	С токеном Moderator	200 OK



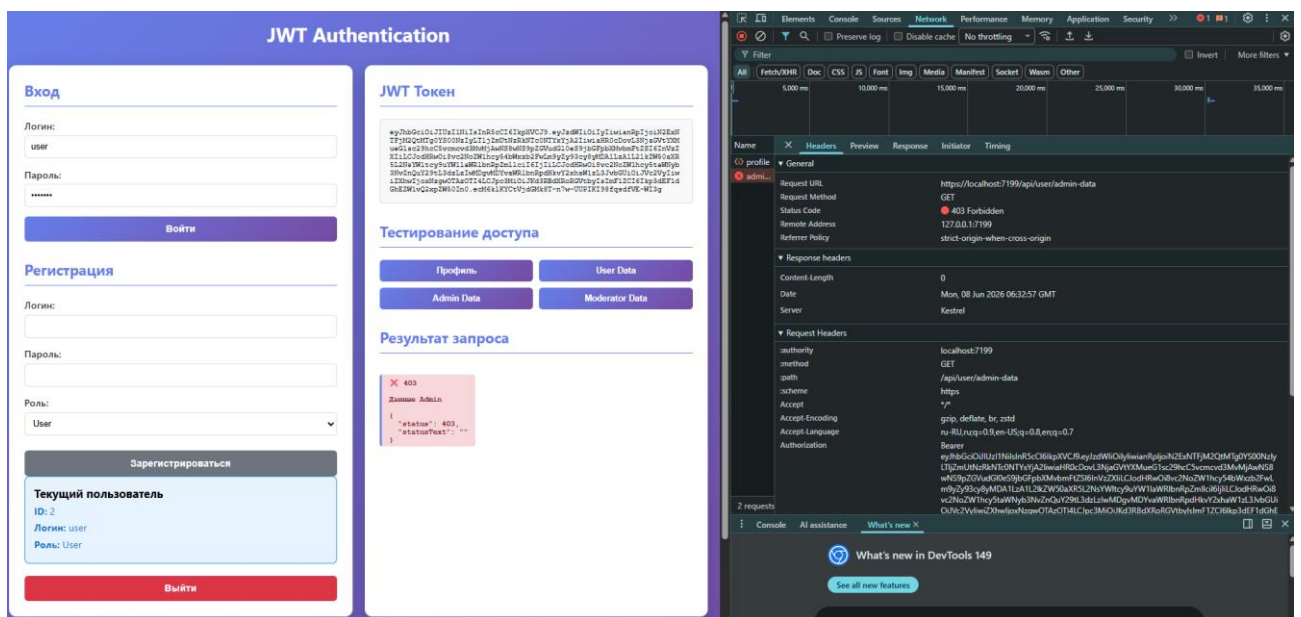
Изображение 3 Успешный вход (POST /api/auth/login) и получение JWT-токена



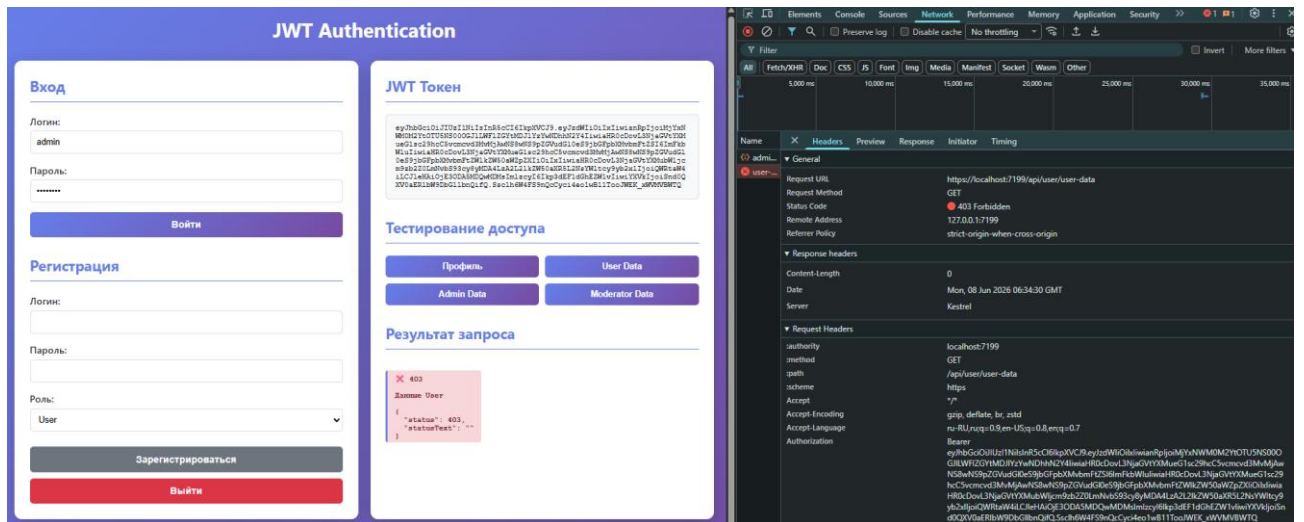
Изображение 4 Запрос без токена: статус 401 Unauthorized



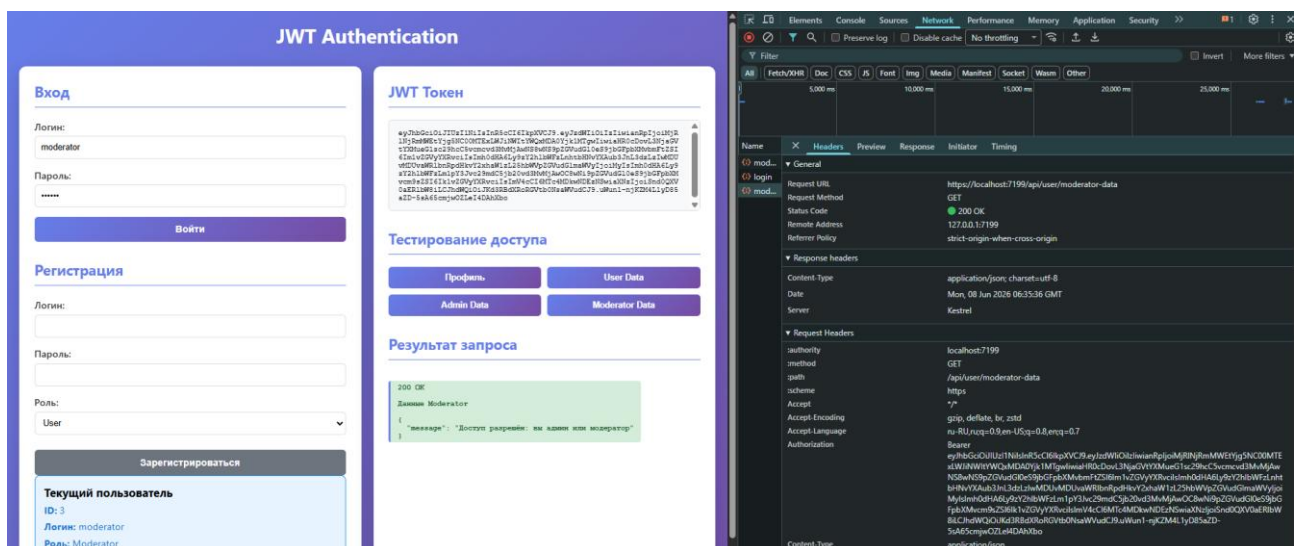
Изображение 5 Запрос с корректным токеном: статус 200 OK



Изображение 6 Запрос с недостаточной ролью: статус 403 Forbidden



Изображение 7 Запрос /api/user/user-data с токеном admin



Изображение 8 Запрос /api/user/moderator-data с токеном moderator

3. Листинг кода

Services/TokenService.cs (генерация токена)

```
public string GenerateToken(User user)
{
    var jwtSettings = _configuration.GetSection("JwtSettings");
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Name, user.Username),
        new Claim(ClaimTypes.Role, user.Role)
    };

    var key = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["SecretKey"]!));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: jwtSettings["Issuer"],
        audience: jwtSettings["Audience"],
        claims: claims,
        expires:
    DateTime.UtcNow.AddMinutes(int.Parse(jwtSettings["ExpirationInMinutes"]!)),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Controllers/AuthController.cs (Выдача токена после проверки пароля)

```
[HttpPost("login")]
public IActionResult Login([FromBody] LoginModel model)
{
    var user = _users.FirstOrDefault(u => u.Username == model.Username);

    // Проверка пароля через BCrypt
    if (user == null || !BCrypt.Net.BCrypt.Verify(model.Password, user.PasswordHash))
        return Unauthorized(new { message = "Неверный логин или пароль" });

    var token = _tokenService.GenerateToken(user);

    return Ok(new { token, userId = user.Id, username = user.Username, role = user.Role
});
}
```

Controllers/UserController.cs (Ограничение доступа атрибутами)

```

[ApiController]
[Route("api/[controller]")]
public class UserController : ControllerBase
{
    // Доступен любому аутентифицированному пользователю
    [HttpGet("profile")]
    [Authorize]
    public IActionResult GetProfile()
    {
        var username = User.FindFirstValue(ClaimTypes.Name);
        var role = User.FindFirstValue(ClaimTypes.Role);
        return Ok(new { message = "Доступ разрешён", username, role });
    }

    // Доступен только администраторам
    [HttpGet("admin-data")]
    [Authorize(Roles = "Admin")]
    public IActionResult GetAdminData() => Ok(new { message = "Секретные данные для админа" });

    // Доступен по кастомной политике
    [HttpGet("moderator-data")]
    [Authorize(Policy = "AdminOrModerator")]
    public IActionResult GetModeratorData() => Ok(new { message = "Доступ для админа или модератора" });
}

```

wwwroot/index.html (Ключевой момент отправки запроса с токеном на JS)

```

async function testEndpoint(endpoint)
{
    const token = localStorage.getItem('jwt_token');
    const response = await fetch(`https://localhost:7001${endpoint}`, {
        method: 'GET',
        headers:
        {
            'Authorization': `Bearer ${token}`, // Передача токена в заголовке
            'Content-Type': 'application/json'
        }
    });
    const data = await response.json();
    console.log(`Статус: ${response.status}`, data);
}

```